SecDV: A Lightweight Secure Deep Neural Network Inference Service With Dynamic Verification

Dong Li, *Graduate Student Member, IEEE*, Anupam Chattopadhyay, *Senior Member, IEEE*, Yang Zhou, Bowen Xing, Qingguo Lü, *Member, IEEE*, Jiahui Wu, Tao Xiang, *Senior Member, IEEE*, and Xiaofeng Liao, *Fellow, IEEE*

***Abstract*—Existing secure deep neural network inference works primarily focus on improving inference efficiency, while largely overlooking the integrity verification of model parameters and tamper detection mechanisms in dynamic scenarios. In this paper, we propose SecDV: a lightweight secure deep neural network inference service with dynamic verification. Specifically, SecDV leverages the CKKS homomorphic encryption scheme to encrypt both input data and model parameters, ensuring their confidentiality. Second, SecDV designs a dynamic verification tree $\mathcal{DVT}$ based on the Chameleon hash function and digital signature mechanism, which achieves three key objectives: (1) accurately identifying tampered model parameters in the presence of a malicious server; (2) supporting legitimate dynamic updates of model parameters; and (3) enabling efficient verification for multiple clients. Security analysis shows that SecDV ensures both data and model confidentiality while providing robust integrity verification and tamper detection. Performance evaluation further demonstrates that SecDV achieves superior efficiency in terms of verification time and storage overhead compared to the existing works.**

***Index Terms*—Deep neural network, security, dynamic verification, chameleon hash, fully homomorphic encryption.**

I. Introduction

In RECENT years, deep neural networks (DNNs) have demonstrated significant advancements, largely attributed to enhanced computational capabilities, the proliferation of large-scale datasets, and continual refinements in model architectures and training methodologies. The core principle of DNNs lies in constructing multiple hidden layers that can learn hierarchical feature representations from extensive training data, thereby enhancing the performance of classification and prediction tasks [1], [2]. Owing to these characteristics, DNNs have been widely applied in various domains, including image classification, financial models, speech recognition, and weather forecasting.

Although DNN inference services provide significant convenience across various domains, the risk of data leakage remains a critical concern [3]. To address this issue, a wide range of secure DNN inference frameworks have been proposed. These works leverage advanced cryptographic techniques such as homomorphic encryption, secure multi-party computation, and trusted execution environments to enable privacy-preserving DNN inference over encrypted data. Collectively, these works have made substantial progress in improving the efficiency, accuracy, and practicality of encrypted inference, thereby facilitating the deployment of privacy-preserving machine learning services in outsourced cloud environments.

For instance, Dowlin et al. [4] proposed CryptoNets, a pioneering method that transformed pre-trained neural networks into models capable of operating directly on encrypted inputs. CryptoNets utilized the YASHE homomorphic encryption scheme [5] to protect both the input data and the model parameters, and approximated the activation function with a low-degree polynomial to maintain compatibility with FHE operations. Building upon this, Chabanne et al. [6] enhanced the CryptoNets framework by integrating batch normalization with polynomial approximation. Their method represented the first privacy-preserving neural network classification scheme that supports models with a depth greater than two. To further improve inference efficiency, particularly addressing the high computational cost associated with FHE, Delphi [7] introduced a hybrid secure inference framework by combining partially homomorphic encryption with a secure two-party computation protocol. In Delphi, the non-linear activation function is evaluated using garbled circuits [8], thereby striking a balance between privacy and performance. More recently, Leia [9] proposed a lightweight privacy-preserving DNN inference framework based on oblivious transfer and secret sharing techniques, aiming to reduce computational and communication overhead while maintaining strong privacy guarantees.

Most existing privacy-preserving inference frameworks assume honest-but-curious cloud servers that faithfully execute

Received 12 May 2025; revised 6 July 2025; accepted 7 August 2025. Date of publication 13 August 2025; date of current version 2 December 2025. This work was supported in part by the Graduate Research and Innovation Foundation of Chongqing under Grant CYB25038, in part by the National Natural Science Foundation of China under Grant U20A20176 and Grant 62072062, and in part by the Natural Science Foundation of Chongqing (Innovation and Development Joint Fund) under Grant CSTB2023NSCQ-LZX0149. Recommended for acceptance by Dr. Yao Sun. *(Corresponding authors: Qingguo Lü; Xiaofeng Liao.)*

Dong Li, Bowen Xing, Qingguo Lü, Tao Xiang, and Xiaofeng Liao are with the Key Laboratory of Dependable Services Computing in Cyber Physical & Society-Ministry of Education, College of Computer, Chongqing University, Chongqing 400044, China (e-mail: lilvmy@163.com; 202214131035t@stu.cqu.edu.cn; lvqingguo886@126.com; txiang@cqu.edu.cn; xfliao@cqu.edu.cn).

Anupam Chattopadhyay is with the College of Computing and Data Science, Nanyang Technological University, Singapore 639798 (e-mail: anupam@ntu.edu.sg).

Yang Zhou is with Tencent Technology Company, Ltd., Guangzhou 510000, China (e-mail: 592191580@qq.com).

Jiahui Wu is with Peng Cheng Laboratory, Shenzhen 518055, China (e-mail: jiahuiwu2022@163.com).

Digital Object Identifier 10.1109/TNSE.2025.3598578

2327-4697 © 2025 IEEE. All rights reserved, including rights for text and data mining, and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.

Authorized licensed use limited to: GUANGZHOU UNIVERSITY. Downloaded on October 19,2026 at 08:58:06 UTC from IEEE Xplore. Restrictions apply.

protocols while attempting to learn private information. This assumption overlooks a critical threat: malicious servers may tamper with model parameters or substitute models entirely, leading to incorrect predictions or degraded performance. Such attacks are particularly dangerous in security-sensitive domains like healthcare, finance, and autonomous systems, where inference errors can lead to severe consequences. Despite these risks, current frameworks rarely verify model integrity, creating a significant security gap where input privacy is protected, but inference results remain vulnerable. This limitation necessitates new secure inference mechanisms that simultaneously ensure data confidentiality and model integrity verification against fully malicious adversaries.

To address the critical challenge of ensuring model integrity in privacy-preserving DNN inference services, we propose SecDV, a lightweight secure DNN inference service with dynamic verification. Specifically, SecDV employs the CKKS [10] homomorphic encryption scheme to encrypt both input data and model parameters, enabling DNN inference while preserving privacy. Meanwhile, to detect the tampering of model parameters by malicious cloud server, SecDV constructs a novel dynamic verification tree $\mathcal{DVT}$ for model parameters based on the Chameleon hash function [11] and digital signature [12]. The framework provides precise identification of tampered parameters, allowing clients to accurately identify which specific model parameters have been maliciously modified even when interacting with an untrusted server.

To implement truly lightweight verification, SecDV introduces a binary processing technique to pre-process model parameters, effectively transforming high-dimensional tensors into scalar representations while preserving their verifiability properties. These compressed model parameters are then encrypted with CKKS and stored as leaf nodes of $\mathcal{DVT}$. Crucially, $\mathcal{DVT}$ incorporates model version information and timestamps into the dynamic verification structure, enabling version control and temporal validation of model parameters. This design not only allows clients to verify the integrity of parameters but also ensures they are using the correct model version at the intended time, preventing potential model downgrade or replay attacks. Moreover, the Chameleon hash properties of $\mathcal{DVT}$ enable legitimate model updates by authorized parties while maintaining verification capabilities, supporting the natural evolution of models through fine-tuning and improvements without rebuilding the entire verification structure. The timestamp and version information are cryptographically bound to the root node of $\mathcal{DVT}$ through digital signatures, creating a tamper-evident chronological record of model updates. Furthermore, SecDV supports efficient batch verification, enabling clients to simultaneously verify multiple model parameters with significantly reduced computational overhead compared to traditional parameter-by-parameter methods. The decentralized nature of our verification mechanism enables multiple independent clients to concurrently verify model integrity without coordination overhead or performance degradation, making SecDV highly scalable for multi-client deployment scenarios. This multi-client verification capability, combined with the system's lightweight design, makes SecDV practical for deployment across a wide range of applications and environments, from resource-constrained IoT devices to large-scale enterprise systems.

The main contributions of SecDV are summarized as follows:

- *Dynamic verification and Tamper Detection:* SecDV introduces $\mathcal{DVT}$, a novel verification structure that combines Chameleon hash functions with digital signatures to enable precise identification of tampered model parameters. This framework uniquely integrates model version and timestamps, creating a tamper-evident chronological record that prevents model downgrade and replay attacks while supporting legitimate model updates by authorized parties without rebuilding the verification structure.
- *Lightweight Parameter Transformation Technique:* SecDV develops a binary processing approach that efficiently transforms high-dimensional model parameters into verifiable scalar representations, dramatically reducing verification overhead while preserving security properties. By encrypting these compressed model parameters as leaf nodes in $\mathcal{DVT}$, SecDV achieves practical verification without compromising the encrypted model or significantly increasing computational burden.
- *Scalable Multiple Clients Batch Verification:* SecDV implements a decentralized verification mechanism that supports both efficient batch verification of multiple parameters simultaneously and concurrent verification by multiple independent clients without coordination overhead. This dual-capability approach significantly reduces computational costs while enabling highly scalable deployments across diverse environments, from resource-constrained IoT devices to enterprise-scale systems.

The rest of this paper is structured as follows. Section II introduces the related works associated with secure and verifiable DNN inference. Section III recalls the corresponding technologies used in SecDV. We give the system architecture and provide a detailed description of SecDV construction in Section IV and Section V, respectively. Sections VII and Section VII present the security and performance analysis, respectively. Finally, Section VIII summarizes the work.

II. Related Works

In this section, we primarily discuss the related works from privacy-preserving DNN inference and model integrity verification, and summarize the key distinctions of some works in Table I.

A. Secure DNN Inference With FHE

Since CryptoNets [4] first proposed a secure CNN inference framework, many researchers have designed works [13], [14], [15], [16] based on CryptoNets to improve the privacy-preserving DNN inference efficiency with different cryptographic techniques. For example, CryptoDL [13] proposed the use of an approximated n-degree polynomial to replace standard activation functions, enabling compatibility with fully homomorphic encryption (FHE). Gelu-Net [14] adopted split learning by decomposing the model into linear and non-linear components: the linear parts are evaluated under FHE, while

Authorized licensed use limited to: GUANGZHOU UNIVERSITY. Downloaded on October 19,2026 at 08:58:06 UTC from IEEE Xplore. Restrictions apply.

TABLE I
COMPARISON OF DIFFERENT PRIVACY-PRESERVING DNN INFERENCE SCHEMES

Scheme	Cryptographic Primitive	Model Privacy	Verification Overhead	Dynamic Update	Tamper Detection
CryptoNets [4]	FHE	✓	✗	✗	✗
CryptoDL [13]	FHE	✓	✗	✗	✗
LoLA [21]	FHE	✓	✗	✗	✗
MUSE [20]	HE+GC+SS	✓	✗	✗	✗
LPFHE [16]	FHE	✓	✗	✗	✗
FHE-DiNN [25]	TFHE	✓	✗	✗	✗
Sonic [31]	SS	✓	✗	✗	✗
ICL-EDI [41]	HS	✗	High	✗	✗
Ateniese *et al.* [42]	HVT	✗	High	✗	✗
Wang *et al.* [43]	AS	✗	High	✗	✗
FMIV [44]	RSA	✗	High	✗	✗
VMHT [45]	PRF	✗	High	✗	✗
ZKCNN [46]	FHE+zk-SNARKS	✓	High	✗	✗
PVCNN [47]	FHE+zk-SNARKS	✓	High	✗	✗
vPIN [48]	FHE+zk-SNARKS	✓	High	✗	✗
SecDV (our work)	**FHE+DS**	✓	**Low**	✓	✓

GC is the garbled circuits, SS is the secret sharing, HS is the homomorphic signature, THFE is the fully homomorphic encryption over the Torus, HVT is the homomorphic verification tag, AS is the arithmetic signature, PRF is the pseudo-random functions and DS is the digital signature, zk-SNARKS is zero-knowledge succinct non-interactive argument of knowledge.
✓: the conditions are met, ✗: the conditions are not met.

the non-linear parts are handled through interactive protocols between the client and the cloud server. LPFHE [16] leveraged RNS-CKKS encryption paired with simplified polynomial constructions to accurately approximate ReLU functionality. Advancing this line of research, the works [17], [18], [19] devised an EVA compiler to automatically encode a vectorized representation into FHE. However, these schemes have a high latency for image classification. To reduce the inference latency, LoLa [21] introduced a specialized FHE packing strategy that significantly reduces the latency of neural network inference. Falcon [22] further improved inference efficiency by leveraging spectral inference techniques [23], [24]. Both schemes just achieved performance gains by redundantly replicating input data within ciphertexts, which minimizes computation overhead. However, these optimizations are particularly advantageous only when the dimensions of neural network layers are substantially smaller than the number of available ciphertext slots, limiting their scalability for deeper or wider models. To achieve secure inference for any depth of the DNN model, the works [25], [26] have explored the use of the TFHE scheme [27] to evaluate discretized neural networks. These schemes benefit from constant computational overhead regardless of network depth. However, their inference latency remains higher than that of prior schemes, largely due to the substantial cost of individual arithmetic operations in TFHE. Moreover, all TFHE-based approaches necessitate careful quantization of both weights and inputs to achieve acceptable efficiency.

B. Secure DNN Inference With Mixed Scheme

Dual-server architectures represent another method for privacy-preserving CNN inference [28], [29], [30]. These schemes distribute either the input data or network weights between two computational entities using secret sharing and secure two-party computation techniques [20], [31], [32], [33], [34], [35]. For example, SecureSL [28] proposed a vertical cooperative inference scheme by using split learning, where they adopted multi-key homomorphic encryption to ensure the confidentiality of the images and model parameters. Manto [29] designed a sliding-window-based fine-tuning method to speed up the encrypted inference. SecureTrain [30] utilized homomorphic secret sharing to support joint linear and non-linear computation in the CNN inference process, aimed to prevent accuracy loss. While improving the DNN inference efficiency over the ciphertexts, these methods remain susceptible to interception by adversaries, as previously mentioned. MUSE [20] demonstrated vulnerability through specially crafted model extraction attacks targeting protocols using additive secret sharing within seemingly secure semi-honest inference systems. Addressing these newfound weaknesses, MUSE incorporated authentication codes for messages, verified secret sharing techniques, and cryptographic zero-knowledge verification methods, but security enhancements that inevitably increase computational overhead through additional homomorphic procedures and communication-intensive proof verification. To solve this issue, Huang et al. [33] emphasized only dividing the data while maintaining public model architecture; alternatively, both Sonic [31] and PCNN_{CEC} [36] extended protection by distributing model components as well. For edge computing environments, Zhao et al. [37] developed optimal security protocols specifically engineered for distributed cloud server deployments. Although these secret-sharing implementations demonstrate excellent computational efficiency, practical deployment faces significant obstacles due to underlying security assumptions and implementation requirements in commercial settings. Overcoming some of these constraints, the recently proposed PCCS framework [38] implements DT-PKC cryptographic methods across dual servers to enhance confidentiality protections. SecureRC [39] designed a privacy-preserving relation classification system, where the attention-based gated recurrent unit was proposed to implement the privacy-preserving relation classification task. Chen et al. [40] developed an authenticable secure multi-party computing algorithm by combining secret sharing, Diffie-Hellman key agreement, and functional encryption technologies, aimed to guarantee the confidentiality of both input data and model parameters, and then provided an authentication tool in the lack of a trusted third party scenarios.

C. Integrity Verification of DNN

To achieve model integrity verification, Cui et al. [41] proposed ICL-EDI, a framework that utilizes homomorphic signature mechanisms to verify model parameter integrity. However, their scheme cannot defend against forgery and replay attacks. Ateniese et al. [42] introduced a provable data possession mechanism based on homomorphic verification tags. Wang et al. [43] provided a data integrity verification framework with algebraic signature, which support data dynamics in outsourced cloud scenarios. However, these schemes exhibited significant latency when applied to model parameter integrity verification tasks. To address these performance limitations, Zhang et al. [44] proposed FMIV, a Merkle Hash Tree (MHT) based methodology for global model parameter integrity verification within federated learning systems. Their scheme partitioned model parameters

Authorized licensed use limited to: GUANGZHOU UNIVERSITY. Downloaded on October 19,2026 at 08:58:06 UTC from IEEE Xplore. Restrictions apply.

into discrete data blocks that serve as leaf nodes in the MHT structure, with RSA encryption providing additional security. Moreover, Li et al. [45] provided a probabilistic data integrity guarantee framework with verifiable Merkle hash tree VMHT, where VMHT generated the integrity proof of each data replica and uniformly sampled data blocks during the audit. While these tree-based schemes successfully reduced verification computational overhead, they nevertheless introduced substantial update costs when implemented in dynamic environments. Another category of model integrity verification is the zero-knowledge proofs approach [46], [47], [48], [49], [50], where a series of arithmetic constraints are employed in arithmetic circuits, i.e., poor selection of model parameters can lead to a significantly bloated proof circuit, making the evaluation process highly inefficient. For example, ZKCNN [46] introduced a novel sumcheck protocol optimization for FFT-based convolutions, achieving linear-time proving and supporting large-scale models such as the VGG16 model. However, the computational overheads of these schemes are extremely large, resulting in inefficient verification. PVCNN [47] developed a novel quadratic matrix program (QMP)-based arithmetic circuit for 2-D convolution operations, significantly reducing proving overhead compared to traditional QAP-based zk-SNARKs. Moreover, multiple proofs from different testers are aggregated into a single proof to further optimize verification. vPIN [48] leveraged partial homomorphic encryption combined with commit-and-prove zk-SNARKs to achieve both privacy-preserving and verifiable CNN inference. By optimizing the proving circuits for homomorphic evaluation, vPIN significantly improved efficiency in proving and verification. vCNN [49] introduced a novel convolution relation representation and zk-SNARK technologies to achieve an efficient verifiable CNN framework that significantly accelerates proving. PPVCNN [50] proposed a privacy-preserving verifiable CNN scheme leveraging collaborative zk-SNARKs based on Bulletproofs, enabling clients to obtain verifiable proofs of classification without revealing their inputs. This approach also supports selective disclosure of classification properties which are verifiable by third parties.

III. Preliminaries

In this section, we primarily recall some technologies used in SecDV, including the CKKS homomorphic encryption, DNN, the Chameleon hash function, and the elliptic curve digital signature.

A. CKKS

The CKKS provides a framework for performing approximate arithmetic operations on encrypted real or complex values, making it particularly suitable for privacy-preserving machine learning and statistical analysis. The CKKS includes 7 algorithms as follows.

- $KeyGen(1^{\lambda})$: Given a security parameter λ, define the initial ciphertext modulus as 2^L, where L is a initial noise budget. Let $\mathbb{R}_{2^L}$ denote the polynomial ring modulo 2^L, and $HWT(h)$ represent a uniform distribution selecting a polynomial from $\mathbb{R}_{2^L}$ with exactly h non-zero coefficients. Sample secret polynomial s from $HWT(h)$, public polynomial a uniformly from $U(\mathbb{R}_{2^L})$, and error polynomial e from Gaussian distribution $DG(\lambda^2)$. Consequently, the secret key is defined as $hsk = (1, s)$, the public key as $hpk = ([-a \cdot s + e]_{2^L}, a)$, and the evaluation key $hvk = ([-a + e' + L \cdot s^2]_{2^L}, a')$, with polynomials a', e' uniformly and Gaussian sampled, respectively.
- $Encrypt(hpk, m)$: To encrypt message polynomial $m \in \mathbb{R}_{2^L}$, a polynomial $v \in U(\mathbb{R}_{2^L})$ is sampled uniformly at random, and error polynomials $e_0, e_1 \in DG(\lambda^2)$ are drawn from a discrete Gaussian distribution. The ciphertext is then generated as follows $ct = ([-v \cdot hpk + (m + e_0)]_{2^L}, e_1)$.
- $Decrypt(sk, ct)$: Given the ciphertext $ct = (c_0, c_1) \in \mathbb{R}_{2^L}$ along with hsk, the plaintext is recovered by evaluating $m = [c_0 + c_1 \cdot hsk]_{2^L}$.
- $Add(ct_1, ct_2)$: For ciphertexts $ct_1 = (c_0, c_1)$ and $ct_2 = (c'_0, c'_1)$, the homomorphic addition operation generates $ct_{Add} = [(c_0 + c'_0, c_1 + c'_1)]_{2^L}$.
- $Mult(ct_1, ct_2)$: For ciphertexts $ct_1 = (c_0, c_1)$ and $ct_2 = (c'_0, c'_1)$, compute intermediate terms

$$(d_0, d_1, d_2) = (c_0 c'_0, c_1 c'_0 + c_0 c'_1, c_1 c'_1)_{2^L}.$$

  The ciphertext multiplication output is $ct_{Mult} = (d_0, d_1) + [2^{-L} \cdot d_2 \cdot hvk]_{2^L}$.
- $Rescale(ct, f)$: Given the ciphertext $ct = (c_0, c_1)$ and an integer scaling factor $f \leq l$, where l denotes the current noise budget of the ciphertext, rescaling reduces the ciphertext modulus and the output is obtained as $ct' = ([2^{-f} \cdot c_0]_{2^L}, c_1)$.
- $Rotate(ct, r)$: Ciphertext vector ct can be cyclically rotated by integer parameter r, resulting in a rotated ciphertext vector ct' shifted rightward by r positions.

B. DNN

DNN, through its multi-layered composition, enables the automatic extraction of increasingly abstract features while maintaining computational efficiency. A standard DNN architecture incorporates the following four primary components:

- *Input Layer:* This layer serves as the interface between external data and the network's internal processing mechanisms. Specifically, for an input matrix with dimension $n \times n$, each element corresponds to a distinct feature representation that undergoes subsequent transformation.
- *Hidden Layers:* These intermediate processing stages execute weighted transformations on incoming signals to extract hierarchical representations. For j-th hidden layer containing k neurons, the output is computed as $y_{i,k} = \sum_{l=0}^{m-1} \sum_{n=0}^{m-1} w_{l,n} x_{j+l,k+n} + b$, where $w_{l,n}$ signifies the convolutional kernel with the dimension $m \times m$, $x_{j+l,k+n}$ represents the activation from the preceding layer, and b denotes the shared bias term.
- *Activation Layers:* These mathematical operations introduce non-linearity into the network's computational graph, enabling the approximation of complex functions. In general, the activation function used is $ReLU = max(0, x)$

Authorized licensed use limited to: GUANGZHOU UNIVERSITY. Downloaded on October 19,2026 at 08:58:06 UTC from IEEE Xplore. Restrictions apply.

while a polynomial approximation is used in SecDV to be compatible with the CKKS.

- *Output Layer:* Neurons in this terminal component translate internal representations into task-specific predictions. For classification tasks with k categories, the layer typically incorporates k computational units with appropriate activation functions, where the computation process is the same as the convolutional process in the hidden layer.

C. Chameleon Hash

A chameleon hash is a trapdoor hash function, i.e, allows someone with trapdoor information to find hash collisions, and consists of the following algorithms.

- $KeyGen$: Given two large primes p and q such that $q|(p-1)$ and let $g \in \mathbb{Z}_p^*$ be a generator of a subgroup of order q. Choose a secret key $csk = x \in \mathbb{Z}_p^*$, and compute the public key $cpk = g^x \mod p$. Define a pre-image resistant hash function $\mathcal{H} : \{0,1\} \to \{0,1\}^l$, where $l \geq \lceil \log_2 p \rceil$.
- $Hash$: Given two random numbers $\alpha, \beta \in \mathbb{Z}_p^*$, compute $e = \mathcal{H}(m||\alpha)$ with a message m. Define the chameleon hash ch as

$$ch(m, \alpha, \beta) = \alpha - (cpk^e g^\beta \mod p) \mod q.$$

- $Forge$: Given a new message $m' \neq m$ and the chameleon hash output $c_{ch} = ch(m, \alpha, \beta)$, one can find a collision by choosing a random number $\delta \in \mathbb{Z}_q^*$. The corresponding collision is then found at (m', α', β'), since

$$\begin{aligned} ch(m', \alpha', \beta') &= \alpha' - (cpk^{e'} g^{\beta'} \mod p) \mod q \\ &= ch(m, \alpha, \beta), \end{aligned}$$

where $\alpha' = c_{ch} + (g^\delta \mod p) \mod q$, $e' = \mathcal{H}(m'||\alpha')$ and $\beta' = \delta - e' \cdot csk \pmod q$.

D. Elliptic Curve Digital Signature

The elliptic curve digital signature algorithm (ECDSA) is a secure digital signature method, which consists of 3 algorithms as follows.

1) $KeyGen$: Given an elliptic curve $E_p(a, b)$ defined over a finite field $\mathbb{F}_p$, a base point G on the curve is selected. The signer chooses a private key $sk \in \mathbb{Z}_n^*$, where n is the order of G, and computes the corresponding public key $pk = sk \cdot G$.
2) $Sign$: To sign a message m, the signer selects a random integer $k < n$, and computes the curve point $P = k \cdot G = (x_1, y_1)$. Let $r = x_1$, the signer computes the message hash $e = hash(m)$ and the signature component $sig = k^{-1}(e + r \cdot sk) \mod n$. The resulting signature is the pair (r, sig). If either r or sig equals zero, repeat the process with a new random number k.
3) Ver: Given a signature (r, sig) and a message m, the verifier computes $e = hash(m), u_1 = sig^{-1}e \mod n, u_2 = sig^{-1}r \mod n, P = u_1 \cdot G + u_2 \cdot pk = (x, y)$. If $r \equiv x' \mod n$, the verifier accepts this signature; otherwise, he/she reject it.

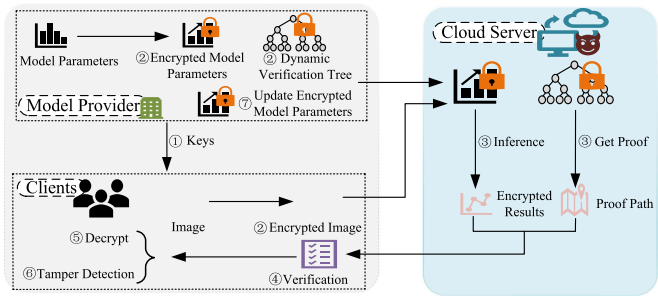


Fig. 1. System model of SecDV.

IV. System Architecture

In this section, we mainly introduce the system model and the threat model. The detailed description is as follows.

A. System Model

As shown in Fig. 1, our SecDV system includes the client, the model provider, and the cloud server, where the client holds a private data and the key pairs of the CKKS scheme, the model provider generates the verification key pairs and signature key pairs, constructs a dynamic verification tree and updates the model parameter when necessary, the cloud server provides the encrypted inference service and generates the proof of the model.

- *Step* ①: The model provider generates the verification public/private key pairs and the signature public/private key pairs, and the client generates the CKKS encryption public/private key pairs. Meanwhile, the client and the model provider share these keys through a secure channel.
- *Step* ②: The model provider employs the CKKS public key to encrypt their model parameters and uses the verification public key and the signature private key to build the dynamic verification tree, then sends them to the cloud server. Moreover, the client uses the CKKS public key to encrypt his/her input image, then sends it to the cloud server for inference.
- *Step* ③: The cloud server implements an inference service with the encrypted model parameters, generates the corresponding proof by using the dynamic verification tree, and returns the encrypted inference results and the proof to the client.
- *Step* ④: The client starts to verify the correctness of the proof from the cloud server by using the verification private keys.
- *Step* ⑤: If Step ④ outputs 1, the client uses the CKKS private key to decrypt the encrypted inference results and reads it.
- *Step* ⑥: If Step ④ outputs 0, the client performs the fine-grained tamper detection, accurately locates which model parameters would be maliciously modified.
- *Step* ⑦: The model sends the update request to the cloud server when he/she wants to update the encrypted model parameters.

Authorized licensed use limited to: GUANGZHOU UNIVERSITY. Downloaded on October 19,2026 at 08:58:06 UTC from IEEE Xplore. Restrictions apply.

TABLE II
THE NOTATIONS OF SOME SYMBOLS

Symbols	Notations
$\mathcal{W}$	The set of models
$\mathbf{W}_i$	The i-th model
$\mathbf{M}, c_{\mathbf{M}}$	The plaintext matrix and ciphertext matrix of the image
$\mathcal{C}$	The ciphertext model parameters set
$\mathcal{R}, \mathcal{M}_{\mathcal{R}}$	The ciphertext/plaintext inference results
$\mathcal{S}$	The tamper detection results
ch	The chameleon hash function
esk, epk	The private/public key of signature
vsk, vpk	The private/public key of verification
hsk, hpk	The private/public keys of CKKS scheme
π_i	The proof of i-th model
$\mathbf{P}_{i,j}, c_{\mathbf{P}_{i,j}}$	The j-th plaintext/ciphertext parameters of i-th model
$\Lambda, \mathcal{DVT}_m, \Phi_m$	The model root nodes, model sub-trees and parameter mappings
Ω_i, Ψ_i	The i-th model leaf nodes and parameter mapping

B. Threat Model

- *Threat Model:* In our SecDV system, the client and the model provider are assumed to be trusted parties, whereas the cloud server is considered a malicious server, which means that the cloud server may manipulate or tamper with arbitrary model parameters uploaded by the model provider or arbitrary input data uploaded by the user to produce incorrect or misleading results. Given these potential threats, the SecDV design explicitly protects the confidentiality and integrity of both the client inputs and the model parameters, ensuring that the malicious cloud server can neither obtain unauthorized access to private data nor alter computational outcomes undetected.
- *Out-of-Scope Attacks:* Assume that the client and model provider are an honest party in this paper; thus, we do not address attacks that attempt to infer the model provider-side model by analyzing input-output pairs, such as model extraction or stealing techniques [51], [52], [53]. In cases where the clients may be malicious, prior works have introduced various protective strategies, including detection of extraction behavior [54], [55], [56], restricting the number of queries [55], identifying abnormal querying patterns [52], and applying differential privacy methods to obscure model outputs [57]. Investigating how these approaches can be adapted and implemented within an encrypted setting is left as a potential direction for future research.

V. PROPOSED SECDV SCHEME

In this section, we first give the notations of some symbols in Table II and then build the efficient verification framework in the dynamic scenario. Finally, we give the detailed constructions of the proposed SecDV scheme.

A. Dynamic Verification Framwork

The SecDV scheme is proposed to enable dynamic integrity verification of model parameters while achieving fine-grained tamper detection. The core challenge lies in constructing a verification framework that effectively integrates these dual capabilities for dynamic operational environments. As the previous works relying on zero-knowledge proofs for model parameters integrity verification face three primary limitations: 1) excessive computational overhead, 2) restriction to static model parameters without update mechanisms, and 3) inability to perform batch verification operations. To overcome these constraints, we present $\mathcal{DVT}$, a novel dynamic verification structure combining Chameleon hash functions with the ECDSA. The detailed construction is as follows.

- $\mathcal{DVT}.KeyGen(1^\lambda) \rightarrow (esk, epk, vpk, vsk, pp)$: Given the security parameter λ, the model provider generates the signature public/private key pairs (epk, esk), the verification public/private key pairs $(vpk = cpk, vsk = csk)$ and public parameters $pp = (p, q, g)$, where $p, q \in \mathbb{Z}_p^*$ are two big primes and g is a generator.
- $\mathcal{DVT}.Build(\mathcal{W}, vpk, esk) \rightarrow \mathcal{DVT}$: The $\mathcal{DVT}$ construction method is shown in Algorithm 1.

1) The provider first flattens the i-th model parameter $\mathbf{P}_{i,j}$, then applies (1) to process each elements in $\mathbf{P}_{i,j}$. After that, the mean value $P_{i,j}$ is computed based on (2), where $\mathbf{P}_{i,j} = [p_{i,j,1}, p_{i,j,2}, \ldots, p_{i,j,h}]$, and H is the total number of elements in the flattened $\mathbf{P}_{i,j}$. Note that replacing $\mathbf{P}_{i,j}$ with $P_{i,j}$ significantly reduces both storage and computational overhead. The model provider then encrypts $P_{i,j}$ as $c_{\mathbf{P}_{i,j}} = CKKS.encrypt(hpk, P_{i,j})$ using the CKKS scheme.

$$\begin{cases} p_{i,j,h} = 1 \text{ if } p_{i,j,h} > 0, \\ p_{i,j,h} = 0 \text{ if } p_{i,j,h} \leq 0. \end{cases} \tag{1}$$

$$P_{i,j} = \frac{1}{H} \sum_{h=1}^{H} p_{i,j,h}. \tag{2}$$

2) The model provider generates a leaf node ν for each encrypted parameter $c_{\mathbf{P}_{i,j}}$ and chooses two random numbers $\nu.\alpha, \nu.\beta \in \mathbb{Z}_p^*$, then the encrypted model parameters are combined with $ID_{\mathbf{W}i}$ and $ID_{\mathbf{P}i,j}$ using (3), where ξ is an encoding function that converts model parameters into bytes and H_1 is a standard hash function, such as SHA-256.

$$\widetilde{c}_{\mathbf{P}_{i,j}} = \xi(H_1(ID_{\mathbf{W}_i}), H_1(ID_{\mathbf{P}_{i,j}}), c_{\mathbf{P}_{i,j}}). \tag{3}$$

Subsequently, the model provider computes the chameleon hash $\nu.\eta$ for leaf nodes using (4).

$$\nu.\eta = ch(\widetilde{c}_{\mathbf{P}_{i,k}}, \nu.\alpha, \nu.\beta, vpk). \tag{4}$$

3) The bottom-up model sub-tree construction process iteratively builds a tree structure by starting with the collection of leaf nodes $\Gamma \leftarrow \Omega_i$ and repeatedly merging them until a single root node emerges. During each iteration, the model provider systematically pairs adjacent nodes from the current layer (e.g., $\Gamma[2l-1]$ and $\Gamma[2l]$), creates parent nodes ζ for each pair, randomly chooses two numbers $\zeta.\alpha, \zeta.\beta \in \mathbb{Z}_p^*$, and computes the chameleon hash $\zeta.\eta$ combining the hash values of both children with these randomization factors using (5). These newly created parent nodes form the next layer Γ' of the model sub-tree, with any unpaired node from an odd-length layer being directly promoted to the next level without modification. The model provider proceeds with the aforementioned

Authorized licensed use limited to: GUANGZHOU UNIVERSITY. Downloaded on October 19,2026 at 08:58:06 UTC from IEEE Xplore. Restrictions apply.

Algorithm 1: Lightweight $\mathcal{DVT}$ for Multiple Models.

Input: Models $\mathcal{W} = \{\mathbf{W}_1, \mathbf{W}_2, \ldots, \mathbf{W}_n\}$, chameleon hash function ch, secret keys esk, vpk, hpk.
Output: Dynamic verification tree $\mathcal{DVT}$.

1 **for** $\boldsymbol{W}_i \in \mathcal{W}$ **do**
2 **for** $\boldsymbol{P}_{i,j} \in \boldsymbol{W}_i$ **do**
3 Flatten $\mathbf{P}_{i,j}$ and do binary processing;
4 Compute the mean value $P_{i,j}$ of $\mathbf{P}_{i,j}$;
5 Encrypt $c_{\mathbf{P}_{i,j}} = CKKS.encrypt(hpk, P_{i,j})$;
6 **end**
7 **end**
8 Initialize $\Lambda \leftarrow \varnothing$, $\mathcal{DVT}_m \leftarrow \varnothing$, $\Phi_m \leftarrow \varnothing$;
9 **for** $i \in (1, 2, \cdots, n)$ **do**
10 Initialize $\Omega_i \leftarrow \varnothing$, $\Psi_i \leftarrow \varnothing$;
11 **for** $k \in (1, 2, \cdots, j)$ **do**
12 Create node ν with $\nu.data \leftarrow c_{\mathbf{P}_{i,k}}$;
13 Choose two random numbers $\nu.\alpha, \nu.\beta \leftarrow \mathbb{Z}_p^*$;
14 Encode the encrypted parameters $\widetilde{c}_{\mathbf{P}_{i,k}} = \xi(H_1(ID_{\mathbf{W}_i}), H_1(ID_{\mathbf{P}_{i,k}}), c_{\mathbf{P}_{i,k}})$;
15 Compute chameleon hash $\nu.\eta = ch(\widetilde{c}_{\mathbf{P}_{i,k}}, \nu.\alpha, \nu.\beta, vpk)$;
16 $\Omega_i \leftarrow \Omega_i \cup \{\nu\}$, $\Psi_i(H_1(ID_{\mathbf{P}_{i,k}})) \leftarrow \nu$;
17 **end**
18 Compute the current level nodes $\Gamma \leftarrow \Omega_i$;
19 **while** $|\Gamma| > 1$ **do**
20 Initialize the next level nodes $\Gamma' \leftarrow \varnothing$;
21 **for** $l \in (1, \lfloor |\Gamma|/2 \rfloor)$ **do**
22 Create parent nodes ζ for children nodes;
23 Choose $\zeta.\alpha, \zeta.\beta \leftarrow \mathbb{Z}_p$;
24 Compute $\zeta.\eta = ch(\Gamma[2l-1].\eta \parallel \Gamma[2l].\eta, \zeta.\alpha, \zeta.\beta, vpk)$;
25 $\Gamma' \leftarrow \Gamma' \cup \{\zeta\}$;
26 **end**
27 **if** $|\Gamma| \bmod 2 = 1$ **then**
28 $\Gamma' \leftarrow \Gamma' \cup \{\Gamma[|\Gamma|]\}$;
29 **end**
30 $\Gamma \leftarrow \Gamma'$;
31 **end**
32 Compute i-th model root node $\omega_i \leftarrow \Gamma[1]$;
33 $\Lambda \leftarrow \Lambda \cup \{\omega_i\}$, $\mathcal{DVT}_m(i) \leftarrow \omega_i$, $\Phi_m(i) \leftarrow \Psi_i$;
34 **end**
35 Compute current model roots nodes $\Gamma \leftarrow \Lambda$;
36 **while** $|\Gamma| > 1$ **do**
37 Repeat lines 18-28 to create a global tree;
38 **end**
39 Compute $\mathcal{DVT}$ root node $\omega = \Gamma[1]$;
40 Compute timestamp τ and generate model verison ϑ;
41 $\sigma = Sign(H_1(\omega.\eta) \parallel \tau \parallel \vartheta, esk)$;
42 **return** $\mathcal{DVT}$ with root ω and signature σ;

operations by setting $\Gamma \leftarrow \Gamma'$ and repeating the process until $|\Gamma| = 1$. At this point, the single remaining node is designated as the model's root node $\omega_i \leftarrow \Gamma[1]$. This root node is subsequently registered in the global collection Λ and associated with its corresponding model index within the $\mathcal{DVT}$ framework. This methodology creates a balanced binary tree with height $\lceil \log_2 n \rceil$ for n parameters, ensuring optimal verification path length while maintaining the integrity chain from individual encrypted parameters to the model root through successive application of the chameleon hash function.

$$\zeta.\eta = ch(\Gamma[2l-1].\eta \parallel \Gamma[2l].\eta, \zeta.\alpha, \zeta.\beta, vpk). \quad (5)$$

4) The model provider merges all of the model sub-tree nodes using the same operations, resulting in a global root node ω. Subsequently, the model provider generates a timestamp τ and model version, signing the root hash based on (6), and outputs the complete $\mathcal{DVT}$ framework.

$$\sigma = Sign(H_1(\omega.\eta) \parallel \tau \parallel \vartheta, esk). \quad (6)$$

- $\mathcal{DVT}.GetProof(H_1(ID_{\mathbf{W}_i})) \rightarrow \pi_i$: Given the model identifier $H_1(ID_{\mathbf{W}_i})$, the cloud server first verifies the existence of the model in the $\mathcal{DVT}$. It then constructs a comprehensive verification proof π_i, which consists of two components: (1) a proof path from the root of the model's sub-tree to the global root, and (2) individual proof paths from each parameter's leaf node to the model's sub-tree root. For each node along these paths, the cloud server records the hash value of the sibling node, along with the corresponding parent node's random values ($\nu.\alpha$ and $\nu.\beta$) used in the chameleon hash computation. Subsequently, the cloud server traverses the $\mathcal{DVT}$ following a binary tree structure, where each non-leaf node has left and right children. The proof π_i also indicates the traversal direction (left or right) at each step. This approach enables efficient verification by allowing the client to reconstruct the hash chain from each parameter's leaf node up to the global root, using only the provided proof nodes and the chameleon hash function. The final proof structure π_i includes all encrypted parameters, their individual proof paths, the model's root hash, the global root hash, a timestamp, version information, and an authoritative signature that authenticates the entire structure.
- $\mathcal{DVT}.Verify(H_1(ID_{\mathbf{W}_i}), \pi_i, epk) \rightarrow 0/1$: Given the model identifier $H_1(ID_{\mathbf{W}_i})$, the model verification package π_i, the verification secret key vsk and the signature public key epk, the client first validates the global root signature σ' using (7),

$$\sigma' = Sign(H_1(\omega.\eta) \parallel \tau \parallel \vartheta, epk) \stackrel{?}{=} \sigma. \quad (7)$$

if (7) holds, the client computes the leaf node hash value using (4), then traverses up $\mathcal{DVT}$ by combining this hash value with sibling nodes hash values according to their left/right positions in the proof path. At each step, parent node hash values are computed using (5) with the corresponding random values. Subsequently, the client validates the path from the model sub-tree root to the global root using the same process. Finally, the client confirms that $\widetilde{c}_{\mathbf{P}_{i,j}}$ is correct and outputs 1 only if the signature is valid, all encrypted parameter hash values match their expected hash values, and the computed path from model sub-tree

Authorized licensed use limited to: GUANGZHOU UNIVERSITY. Downloaded on October 19,2026 at 08:58:06 UTC from IEEE Xplore. Restrictions apply.

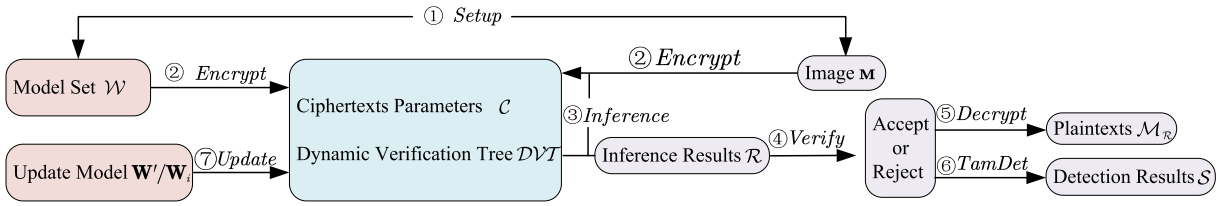


Fig. 2. Overview of SecDV.

root to global root produces the expected global root hash value; otherwise outputs 0.

- $\mathcal{DVT}.Update(flag) \rightarrow \mathcal{DVT}'$: The update algorithm includes 3 operations, i.e., add new model, delete model and modify model parameters.
  1) $\mathcal{DVT}.Add(\mathcal{DVT}, \mathbf{W}'_i) \rightarrow \mathcal{DVT}'$: When adding a new model $\mathbf{W}'_i$, the model provider first creates leaf nodes for each parameter $\mathbf{P}'_{i,j}$ of $\mathbf{W}'_i$, generating random values and computing initial chameleon hash value by using (4). Then the model provider builds a sub-tree for $\mathbf{W}'_i$ by using $\mathcal{DVT}.Build$ algorithm and stores the resulting model root node in the model mapping. Finally, the model provider combines all current model root nodes into a list to send to the cloud server. Note that the model provider records and outputs hash value changes, ensuring a complete traceable path from parameter leaf nodes to the global root.
  2) $\mathcal{DVT}.Delete(\mathcal{DVT}, \mathbf{W}_i, vsk) \rightarrow \mathcal{DVT}'$: The model provider first creates an empty parameter node, computes its chameleon hash value by using (4), and then updates the path from the model root to the global root using $forge(vsk, \cdot)$. During this process, the model provider recalculates random values for each node on the path to produce the same result as the original hash values, even though the underlying data has changed. Finally, the model provider sends all updated nodes to the cloud server. Note that this method preserves the structural integrity of the tree while logically "deleting" the model.
  3) $\mathcal{DVT}.Modify(\mathcal{DVT}, \mathbf{P}_{i,j}, \mathbf{P}'_{i,j}, vsk) \rightarrow \mathcal{DVT}'$: The model provider first replaces the original parameter $\mathbf{P}_{i,j}$ with the new parameter $\mathbf{P}'_{i,j}$, and computes its new chameleon hash value by using (4). Subsequently, the provider updates the entire path from the modified parameter node to the global root using $forge(vsk, \cdot)$. During this process, the model provider recalculates random values for each affected node in the path to ensure that they produce identical hash values as before, despite the underlying parameter data being changed. This collision-finding operation leverages the trapdoor property of the chameleon hash function accessible only through vsk. Finally, the model provider sends all updated nodes to the cloud server. Note that this method preserves the global root hash and the structural integrity of the tree while effectively modifying the target model parameters.

B. Detailed Construction

SecDV includes 7 algorithms, i.e., $Setup$, $Encrypt$, $Inference$, $Verify$, $Decrypt$, $TamDet$ and $Update$, as shown in Fig. 2. We give a detailed description of SecDV as follows.

- $Setup(1^{\lambda})$: The model provider first generates the signature private/public key (esk, spk), verification private/public key $(vpk = cpk, vsk = csk)$ and the public parameters $pp = (p, q, g)$ by recalling $\mathcal{DVT}.KeyGen$ algorithm and shares these secret keys with the client through a secure channel. The client generates the homomorphic encryption private/public key (hpk, hsk) by using the CKKS scheme, and then broadcasts hpk and keeps hsk by himself.
- $Encrypt(hpk, ID_{\mathbf{W}_i}, \mathbf{M}, \mathcal{W}, esk, vpk)$: The encryption process consists of 3 steps to generate the encrypted image $c_{\mathbf{M}}$, the dynamic verification tree $\mathcal{DVT}$, and the encrypted model parameters $c_{\mathbf{P}_{i,j}}$. These steps are described as follows.
  1) Given an image $\mathbf{M}$, the client encrypts $\mathbf{M}$ as $c_{\mathbf{M}} = CKKS.encrypt(hpk, \mathbf{M})$, then computes hash value $H_1(ID_{\mathbf{W}_i})$ of $ID_{\mathbf{W}_i}$. Finally, the client sends $c_{\mathbf{M}}, H_1(ID_{\mathbf{W}_i})$ to the cloud server.
  2) The model provider recalls $\mathcal{DVT}.Build$ algorithm to build the dynamic verification tree $\mathcal{DVT}$ and uses hpk to encrypt their stored model parameters as $c_{\mathbf{P}_{i,j}}$[1], then they send $\mathcal{DVT}, c_{\mathbf{P}_{i,j}}$ to the cloud server.
- $Inferece(c_{\mathbf{M}}, c_{\mathbf{P}_{i,j}}, \mathcal{DVT})$: When the cloud server receives the encrypted image $c_{\mathbf{M}}$ from the client, it immediately runs DNN inference operations over the ciphertext. Moreover, the cloud server recalls $\mathcal{DVT}.GetProof$ algorithm to generate proof π_i for sepcified model $H_1(ID_{\mathbf{W}_i})$. Finally, the cloud server returns the final encrypted results $\mathcal{R}$ and proof π_i to the client.
- $Verify(ID_{\mathbf{W}_i}, \mathcal{DVT}, \pi_i, vpk)$: After receiving the proof π_i from the cloud server for the specified model $H_1(ID_{\mathbf{W}_i})$, the client recalls $\mathcal{DVT}.Verify$ algorithm to verify the integrity of the model parameters.
- $Decrypt(\mathcal{R}, hsk)$: If $\mathcal{DVT}.Verify$ algorithm outputs 1, the client decrypts $\mathcal{R}$ using hsk to read the decrypted results $\mathcal{M}_{\mathcal{R}}$; otherwise, the client rejects $\mathcal{R}$.
- $TamDet(\mathcal{DVT}, hpk, ID_{\mathbf{W}_i}, \tilde{c}_{\mathbf{P}_{i,j}})$: If the cloud server is malicious, it would tamper with the model parameters, resulting in incorrect inference results. The malicious cloud server first generates a multi-dimensional matrix $\mathbf{P}'_{i,j}$ for the specified model parameter $\mathbf{P}_{i,j}$, then uses (1–4) to compute its chameleon hash value as $\tilde{c}'_{\mathbf{P}_{i,j}}$ and replaces the leaf nodes $\tilde{c}_{\mathbf{P}_{i,j}}$ with $\tilde{c}'_{\mathbf{P}_{i,j}}$ in the $\mathcal{DVT}$,

[1]Note that this step does not flatten model parameter $\mathbf{P}_{i,j}$.

Authorized licensed use limited to: GUANGZHOU UNIVERSITY. Downloaded on October 19,2026 at 08:58:06 UTC from IEEE Xplore. Restrictions apply.

recalls $\mathcal{DVT}.GetProof$ algorithm to generate the proof π_i' of the tampered model $H_1(ID_{\mathbf{W}_i})$ and returns π_i' to the client. After receiving the proof π_i', the client performs fine-grained (parameter level) tamper detection by comparing a model's current parameters against the client's locally requested trusted copy from the model provider. For a model $ID_{\mathbf{W}_i}$ and its parameters $\mathbf{W}_i = \{\mathbf{P}_{i,1}, \mathbf{P}_{i,2}, \ldots, \mathbf{P}_{i,n}\}$, let $ch(\widetilde{c}_{\mathbf{P}_{i,j}}, \alpha, \beta, vpk)$ represent the chameleon hash value of the encrypted parameter $\widetilde{c}_{\mathbf{P}_{i,j}}$ computed with ramdom values α, β using the verification public key vpk. Subsequently, the client iterates through each encrypted parameter $\widetilde{c}_{\mathbf{P}_{i,j}}$ from π_i' and compares the current encrypted parameter value $\widetilde{c}_{\mathbf{P}_{i,j}^{\text{current}}}$ with the expected value $\widetilde{c}_{\mathbf{P}_{i,j}^{\text{expected}}}$ stored in his/her local database. A parameter is flagged as modified if $\widetilde{c}_{\mathbf{P}_{i,j}^{\text{current}}} \neq \widetilde{c}_{\mathbf{P}_{i,j}^{\text{expected}}}$, which would result in different chameleon hash values: $ch(\widetilde{c}_{\mathbf{P}_{i,j}^{\text{current}}}, \alpha, \beta, vpk) \neq ch(\widetilde{c}_{\mathbf{P}_{i,j}^{\text{expected}}}, \alpha, \beta, vpk)$ when using the same random values. For each modified parameter, the client records a triplet $(\widetilde{c}_{\mathbf{P}_{i,j}}, \widetilde{c}_{\mathbf{P}_{i,j}^{\text{expected}}}, \widetilde{c}_{\mathbf{P}_{i,j}^{\text{current}}})$ in the result set

$$\mathcal{S}=\{(\widetilde{c}_{\mathbf{P}_{i,j}}, \widetilde{c}_{\mathbf{P}_{i,j}^{\text{expected}}}, \widetilde{c}_{\mathbf{P}_{i,j}^{\text{current}}}) | \widetilde{c}_{\mathbf{P}_{i,j}^{\text{current}}} \neq \widetilde{c}_{\mathbf{P}_{i,j}^{\text{expected}}}\}_{\mathbf{P}_{i,j} \in \mathbf{W}_i}.$$

Note that this direct parameter comparison provides an efficient complement to the $\mathcal{DVT}$ verification process, allowing the clients to immediately identify where specific parameters have been altered.

- $Update(flag, \mathbf{W}_i'/(\mathbf{W}_i, vsk)/\mathbf{P}_{i,j}', \mathcal{DVT})$: The update process adheres to the procedure defined by the $\mathcal{DVT}.Update$ algorithm. Upon receiving an update signal $flag \in \{\text{add}, \text{delete}, \text{modify}\}$, the model provider invokes the corresponding sub-algorithm—$\mathcal{DVT}.Add$, $\mathcal{DVT}.Delete$, or $\mathcal{DVT}.Modify$ to perform the necessary modifications on the nodes within $\mathcal{DVT}$. Subsequently, the updated nodes are transmitted to the cloud server, which updates the $\mathcal{DVT}$ accordingly.

VI. Security Analysis

In this section, we mainly formalize the security properties that the DVT must satisfy through rigorous definitions and proofs. The DVT has hierarchical verification and dynamic update capabilities while preserving verifiability. Meanwhile, both the input data of the client and the model parameters of the model provider are well protected because the private key hsk of the CKKS is only maintained by the client. Furthermore, we also give a theoretical computational complexity analysis.

A. Verification Analysis

Given a game-based scenario between a challenger $\mathcal{B}$ and an adversary $\mathcal{A}$, the $\mathcal{DVT}$ should satisfy structure-preserving and one-wayness characteristics. For structure-preserving, $\mathcal{A}$ can not alter the structure of $\mathcal{DVT}$, i.e., modify the leaf nodes without the assistance of the model provider, where deleting leaf nodes can be seen as modifying the leaf nodes to null. For one-way, $\mathcal{A}$ can not add any new leaf nodes (model parameters) to the $\mathcal{DVT}$. The game-based scenario between the challenger and $\mathcal{A}$ is as follows.

- Initialize: $\mathcal{B}$ generates the verification key pair (vpk, vsk) and the signature key pair (epk, esk), then sends (vpk, epk) to $\mathcal{A}$.
- Uploading: $\mathcal{A}$ chooses the encrypted model paratemer $\widetilde{c}_{\mathbf{P}_{i,j}}$ to $\mathcal{B}$, then $\mathcal{B}$ inserts $\widetilde{c}_{\mathbf{P}_{i,j}}$ into $\mathcal{DVT}$ and computes corrsponding proof $\widetilde{\pi}$ and the root node signature $\widetilde{\sigma}$, sends $(i, \widetilde{\pi}, \widetilde{\sigma})$ to $\mathcal{A}$. Moreover, $\mathcal{A}$ adaptively chooses $(k, \widetilde{c}'_{\mathbf{P}_{i,k}})$ to $\mathcal{B}$, then $\mathcal{B}$ modifies $\widetilde{c}_{\mathbf{P}_{i,k}}$ to $\widetilde{c}'_{\mathbf{P}_{i,k}}$ and sends the updated proof π', the verification public key vpk' and root node signature σ' to $\mathcal{A}$. Finally, $\mathcal{A}$ generates response sequence $Q = \{(\widetilde{c}_{\mathbf{P}_{1,j}}, 1, \widetilde{\pi}_1), \ldots, (\widetilde{c}_{\mathbf{P}_{n,j}}, n, \widetilde{\pi}_n), \widetilde{\sigma}\}$ and the corresponding vpk^*, epk^*.
- Output: $\mathcal{A}$ outputs $(\widetilde{c}^*_{\mathbf{P}_{i,j}}, i^*, \widetilde{\pi}^*, \sigma^*)$.
  1) $Verify(epk^*, vpk^*, i^*, \widetilde{c}^*_{\mathbf{P}_{i,j}}, \widetilde{\pi}^*, \sigma^*) = 1$ if $1 \leq i^* \leq n \wedge (\widetilde{c}^*_{\mathbf{P}_{i,j}}, i^*, \widetilde{\pi}^*, \sigma^*) \notin Q$.
  2) $Verify(epk^*, vpk^*, i^*, \widetilde{c}^*_{\mathbf{P}_{i,j}}, \widetilde{\pi}^*, \sigma^*) = 1$ if $i^* > n$.

We denote $\Pr_{\mathcal{A}}^{sp}$ as the probability that $\mathcal{A}$ wins the game in the structure-preserving situation. Similarity, $\Pr_{\mathcal{A}}^{ow}$ is the probability that $\mathcal{A}$ wins the game in the one-wayness situation. Therefore, we give two security definitions as follows.

Definition 1: If $\Pr_{\mathcal{A}}^{sp}$ is negligible for any $\mathcal{A}$, the dynamic verification tree $\mathcal{DVT}$ is structure-preserving.

Definition 2: If $\Pr_{\mathcal{A}}^{ow}$ is negligible for any $\mathcal{A}$, the dynamic verification tree $\mathcal{DVT}$ is one-way.

Moreover, we define two weaker definitions of $\mathcal{DVT}$ security, where $\mathcal{A}$ is required to commit its queries ahead of receiving the public key.

- Queries: $\mathcal{A}$ generates a sequence of queries $(\widetilde{c}_{\mathbf{P}_{1,j}}, \ldots, \widetilde{c}_{\mathbf{P}_{n,j}})$.
- Response: $\mathcal{B}$ computes the verification key pair (vpk, vsk) and signature key pair (epk, esk). For $1 \leq i \leq n$, $\mathcal{B}$ inserts $\widetilde{c}_{\mathbf{P}_{i,j}}$ into $\mathcal{DVT}$, computes the corresponding proof $\widetilde{\pi}_i$ and the root node signature $\widetilde{\sigma}$, and then returns $\{vpk, \widetilde{\sigma}, (1, \widetilde{\pi}_1), \ldots, (n, \widetilde{\pi}_n)\}$ to $\mathcal{A}$. Meanwhile, $Q = \{(\widetilde{c}_{\mathbf{P}_{1,j}}, 1, \widetilde{\pi}_1), \ldots, (\widetilde{c}_{\mathbf{P}_{n,j}}, n, \widetilde{\pi}_n), \widetilde{\sigma}\}$ is the ordered query/response pairs.
- Output: $\mathcal{A}$ outputs a tuple $(\widetilde{c}^*_{\mathbf{P}_{i,j}}, i^*, \pi^*, \sigma^*)$.
  1) $Verify(epk, vpk, i^*, \widetilde{c}^*_{\mathbf{P}_{i,j}}, \widetilde{\pi}^*, \sigma^*) = 1$ if $1 \leq i^* \leq n \wedge (\widetilde{c}^*_{\mathbf{P}_{i,j}}, i^*, \widetilde{\pi}^*, \sigma^*) \notin Q$.
  2) $Verify(epk, vpk, i^*, \widetilde{c}^*_{\mathbf{P}_{i,j}}, \widetilde{\pi}^*, \sigma^*) = 1$ if $i^* > n$.

We denote $\Pr_{\mathcal{A}}^{w-sp}$ as the probability that $\mathcal{A}$ wins the game in the structure-preserving situation. Similarity, $\Pr_{\mathcal{A}}^{w-ow}$ is the probability that $\mathcal{A}$ wins the game in the one-wayness situation. Therefore, we give two security definitions as follows.

Definition 3: If $\Pr_{\mathcal{A}}^{w-sp}$ is negligible for any $\mathcal{A}$, the dynamic verification tree $\mathcal{DVT}$ is weakly structure-preserving.

Definition 4: If $\Pr_{\mathcal{A}}^{w-ow}$ is negligible for any $\mathcal{A}$, the dynamic verification tree $\mathcal{DVT}$ is weakly one-way.

Authorized licensed use limited to: GUANGZHOU UNIVERSITY. Downloaded on October 19,2026 at 08:58:06 UTC from IEEE Xplore. Restrictions apply.

Theorem 1: If ch is an invertible one-way collision-resistant chameleon hash function, we can say $\mathcal{DVT}$ is secure, including structure-preserving and one-wayness.

Proof: The proof proceeds in three stages. First, we consider the case where $\mathcal{A}$ commits to all inputs before learning the public key of $\mathcal{DVT}$. Second, we allow $\mathcal{A}$ to adaptively choose the encrypted model parameters, excluding modification of the encrypted model parameters. Finally, we permit $\mathcal{A}$ to perform modifications of the encrypted model parameters. These cases are addressed by Lemmas (1)-(3), respectively.

Lemma 1: $\mathcal{DVT}$ is weakly secure, including weakly structure-preserving and weakly one-wayness, if ch is an invertible chameleon hash function with one-wayness and collision resistance.

Proof: To prove the lemma, we employ a proof by contradiction, i.e, $Pr_{\mathcal{A}}^{w-sp}$ is non-negligible in the structure-preserving situation. Therefore, the probability that a collision is found in ch is $\epsilon_{ch}(\lambda)$, respectively. If $\epsilon_{ch}(\lambda)$ is non-negligible, ch is not collision-resistant, which is contrary to the definition of ch, thus, $\epsilon_{ch}(\lambda)$ is negligible. Consequently, $Pr_{\mathcal{A}}^{w-sp}$ is negligible in the structure-preserving situation.

Moreover, we also assume that $Pr_{\mathcal{A}}^{w-ow}$ is non-negligible in the one-wayness situation, i.e., the probability of $\mathcal{A}$ finding the collision of ch is $Pr_{\mathcal{A}}^{w-ow}/t+1$, where $t+1$ is the internal nodes when building $\mathcal{DVT}$. In other words, $\mathcal{A}$ has a considerable probability of finding collisions in ch. Therefore, $\mathcal{A}$ finds collisions with non-negligible probability, which directly violates the one-wayness collision-resistance of ch.

Lemma 2: Given that $\mathcal{DVT}$ is weakly secure, including structure preservation and one-wayness, and ch is a collision-resistant chameleon hash, embedding a chameleon hash output at each leaf node using an independent key is sufficient to achieve full structure-preserving and one-way security for $\mathcal{DVT}$.

Proof: We denote $(\widetilde{c}^*_{\mathbf{P}_{i,j}}, i^*, \pi^*, \sigma^*)$ as the output that $\mathcal{A}$ is against the $\mathcal{DVT}$ security properties, including structure-preserving and one-wayness, where $\pi^* = \pi_i^*, r^*$ are the proof and random number of the i^*-th encrypted model parameter leaf node in weakly secure $\mathcal{DVT}$, respectively. Let $y^* = ch(\widetilde{c}^*_{\mathbf{P}_{i,j}}, r^*)$ and $(y_1, 1, \pi_1), \ldots, (y_n, n, \pi_n)$ be the state of $\mathcal{DVT}$ following the adpative addition of n model parameters by $\mathcal{A}$. Meanwhile, we consider a tuple $(\widetilde{c}_{\mathbf{P}_{i^*,j}}, i^*, \widetilde{\pi}^*_i)$ initially stored at the i^*-th encrypted model parameter leaf node in $\mathcal{DVT}$. We assume $Pr_{\mathcal{A}^{w-sp}}$ and $Pr_{\mathcal{A}^{w-ow}}$ are non-negligible, then we analyze two possible cases as follows.

- If $y_{i^*} = y^*$, then $\mathcal{A}$ has found two dinstinct inputs $(\widetilde{c}_{\mathbf{P}_{i^*,j}}, r_{i^*}) = (\widetilde{c}^*_{\mathbf{P}_{i,j}}, r^*)$ such that
$$ch(\widetilde{c}_{\mathbf{P}_{i^*,j}}, r_{i^*}) = ch(\widetilde{c}^*_{\mathbf{P}_{i,j}}, r^*),$$
violating the collision resistance of the underlying $\mathcal{DVT}$.
- If $y_{i^*} \neq y^*$, then $\mathcal{A}$ has constructed a new value that passes verification using an unchanged proof path, thereby breaking the weak structure-preserving and weak one-wayness properties of the weakly secure $\mathcal{DVT}$.

In either scenario, we can construct an efficient adversary against the assumed hard problem (either collision resistance of ch or weak $\mathcal{DVT}$ security), contradicting the underlying assumptions. Therefore, the success probability of $\mathcal{A}$ must be negligible.

Lemma 3: $\mathcal{DVT}$ is resilient to adaptively modify operations.

Proof: Assume that $\mathcal{A}$ attempts to modify the encrypted model parameters within $\mathcal{DVT}$. Upon each such attempt, a fresh random number r_0 is chosen and stored as part of the public key. We require that $\mathcal{A}$ has not previously queried or observed any outputs of ch instantiated with r_0. Under this assumption, we can define a new hash value deterministically. For example, given two encrypted model parameters $\widetilde{c}_{\mathbf{P}_{i,1}}$ and $\widetilde{c}_{\mathbf{P}_{i,2}}$, the corresponding node value can be computed as $ch(\widetilde{c}_{\mathbf{P}_{i,1}} || \widetilde{c}_{\mathbf{P}_{i,2}}, r_0)$. Since r_0 is chosen randomly and the $\mathcal{DVT}$ has one-wayness and collision-resistance, the output of $ch(\widetilde{c}_{\mathbf{P}_{i,1}} || \widetilde{c}_{\mathbf{P}_{i,2}}, r_0)$ is computationally indistinguishable from the hash value of a random number.

As a result, with overwhelming probability, the computed value will not collide with any existing node in the tree. If a collision occurs with non-negligible probability, it would imply the existence of an adversary capable of breaking the collision-resistance of the underlying ch, thereby violating the assumed hardness of the problem. Therefore, any modification performed by $\mathcal{A}$ without knowledge of the trapdoor information will succeed with at most negligible probability. It guarantees that all unauthorized modifications to the structure are infeasible, thus preserving the integrity of the $\mathcal{DVT}$.

Based on Lemmas 1 to 3, the existence of a probabilistic polynomial-time adversary capable of either altering the structure of $\mathcal{DVT}$ or inserting new encrypted model parameters into leaf nodes in $\mathcal{DVT}$ leaves with non-negligible success probability would imply a violation of either the collision-resistance of the underlying hash function or the security properties (invertibility, one-wayness, or collision-resistance) of ch. Such an outcome would contradict the definition of ch. Consequently, we conclude that the $\mathcal{DVT}$ is secure against $\mathcal{A}$ attack, thereby ensuring the verifiability of the model parameters.

B. Theoretical Efficiency Analysis

Given a security parameter λ, the number m of model parameters and the depth l of DNN, we can derive the computational complexity of all algorithms in SecDV.

- $Setup$: In this algorithm, it mainly generates the homomorphic key pair and the signature key pair, thus, the computational complexity is $\mathcal{O}(\lambda^2 \log^3 \lambda) + \mathcal{O}(\lambda^3)$. It does not affect the efficiency of SecDV because it is one-time.
- $Encrypt$: The computational complexity mainly consists of encrypting model parameters and building DVT, where the computational costs of encrypting model parameters and building $\mathcal{DVT}$ are $\mathcal{O}(m \cdot \lambda \log^2 \lambda)$ and $\mathcal{O}(\log m)$, respectively. Since these operations are one-time, it does not affect the efficiency of SecDV.
- $Inference$: The cloud server runs secure inference operations to generate the encrypted inference results, the computational complexity is $\mathcal{O}(l \cdot \lambda^2 \log^3 \lambda)$, then goes through $\mathcal{DVT}$ to generate corresponding proof, the computational complexity is $\mathcal{O}(\log m)$.

Authorized licensed use limited to: GUANGZHOU UNIVERSITY. Downloaded on October 19,2026 at 08:58:06 UTC from IEEE Xplore. Restrictions apply.

- $Verify$: The client verifies the integrity of the specified model parameters with the returned proof, and the computational complexity is $\mathcal{O}(\log m + 1)$, where the cloud server returns at almost m nodes and a root node.
- $TamDet$: This algorithm is split into two parts. First, the malicious cloud server attempts to tamper with a model parameter, it uses CKKS to encrypt its randomly generated data, so the computational complexity is $\mathcal{O}(\lambda \log^2 \lambda)$, then returns corrsponding proof to the client. Second, the client performs tamper detection operations based on returned proof, the computational complexity is $\mathcal{O}(\log m + 1)$.
- $Update$: The model provider performs update operations for $\mathcal{DVT}$, so the computational complexity is $\mathcal{O}(\log m)$.

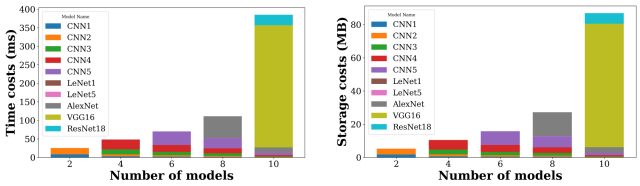
VII. Performance Analysis

In this section, we evaluate the computational and storage overheads of SecDV. Specifically, we analyze the time and storage costs associated with building $\mathcal{DVT}$, verifying model integrity (including scenarios where the cloud server does not perform a tamper operation or the cloud server attempts to tamper with a model parameter or performs batch verifications), and updating $\mathcal{DVT}$ on the cloud server. Furthermore, we provide a comparative analysis with two existing schemes (FMIV [44] and VMHT [45]) to demonstrate the efficiency advantages of SecDV.

Configuration: All experiments in SecDV are conducted in Python on a machine equipped with an Intel(R) Core(TM) i5-12600KF CPU (4.9 GHz) and 32 GB of RAM in Ubuntu. Meanwhile, we utilize the Pyfhel library [58] to implement the CKKS encryption scheme and the python-ecdsa library [59] to generate a digital signature for the root node of $\mathcal{DVT}$. For model integrity verification, we adopt two standard MNIST datasets [60] and CIFAR10 datasets [61] to perform the training process to obtain the model parameters on various DNNs. Specifically, we choose five custom CNNs [48], denoted as CNN1, CNN2, CNN3, CNN4, and CNN5, and four widely used DNNs, i.e., LeNet-1, LeNet-5, AlexNet, VGG16, and ResNet-18,[2] where each of the 5 custom CNNs shares a common architecture comprising a single convolutional layer, one activation layer, a pooling layer, and two fully connected layers, the convolutional layers of all CNNs use a 3×3 kernel, but pooling layers in CNN1 and CNN2 use a 4×4 window size while CNN3, CNN4 and CNN5 use 2×2 window size. The code is available at https://github.com/lilvmy/Dynamic_Verification_PPML.

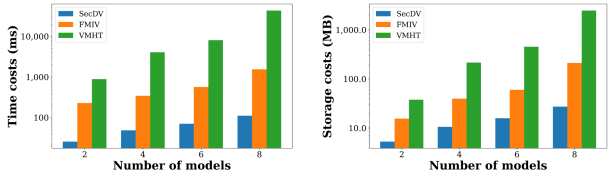
Build $\mathcal{DVT}$: Fig. 3 illustrates both the time and storage costs associated with building $\mathcal{DVT}$ using various numbers of models. As shown in Fig. 3(a) and (b), both time and storage costs increase proportionally with the number of models incorporated. Notably, the inclusion of VGG18 and ResNet18 into $\mathcal{DVT}$ resulted in substantial increases in both time and storage demands. This significant jump can be attributed to the vast differences in parameter counts across the various deep neural networks in our configuration. For instance, LeNet1 contains merely 3,246 parameters, whereas VGG16 encompasses approximately 130 million parameters (a difference of several orders of magnitude).

[2]We trained CNN1, CNN2, CNN3, CNN4, CNN5, LeNet1, and LeNet5 based on MNIST datasets, while AlexNet, VGG16 and ReNet18 are trained based on CIFAR10 datasets.



(a) The time costs of building $\mathcal{DVT}$. (b) The storage costs of building $\mathcal{DVT}$.

Fig. 3. Time costs and storage costs of building $\mathcal{DVT}$ with different numbers of models.



(a) The time costs of building a different verification tree. (b) The storage costs of building a different verification tree.

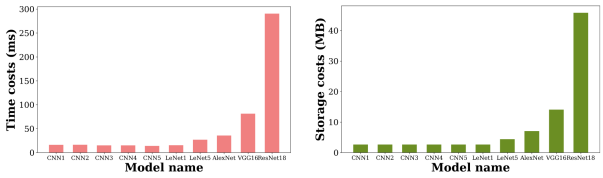
Fig. 4. The comparison of time and storage costs of building different verification tree structures over various numbers of models.

Furthermore, we conduct a comparative analysis of time and storage costs across different verification frameworks under various schemes. As depicted in Fig. 4(a) and (b), while the time and storage costs of SecDV, FMIV, and VMHT all increase proportionally with the number of incorporated models, SecDV consistently demonstrates superior efficiency with significantly lower costs than both FMIV and VMHT, e.g., when the number of models is 6, the time and storage costs of SecDV are 70.214 ms and 15.8 MB, respectively, whereas FMIV is 565.416 ms and 59.704 MB, and VMHT is 8020.576 ms and 451.276 MB. This performance advantage stems primarily from SecDV's use of binary compression techniques, as detailed in the $\mathcal{DVT}.Build$ algorithm. Unlike conventional approaches, SecDV transforms high-dimensional model parameters into scalar values that serve as leaf nodes within $\mathcal{DVT}$. In contrast, both FMIV and VMHT employ a more resource-intensive approach that splits high-dimensional data into numerous data blocks to serve as leaf nodes in their respective verification frameworks.

Verify Normal Models: Fig. 5 illustrates that both the time and storage costs increase significantly as the client requests verification for different unmodified models (i.e., models whose parameters have not been tampered with by the malicious cloud server). As shown in Fig. 5(a) and (b), the verification time and storage overhead for LeNet1 are notably lower compared to other models, where the verification time and storage costs for LeNet1 are 15.328 ms and 2.642 MB, respectively. This is primarily due to its relatively smaller number of parameters than other models.

Furthermore, we compare the verification time and storage overhead of SecDV with those of FMIV and VMHT. As illustrated in Fig. 6(a) and (b), SecDV consistently achieves lower

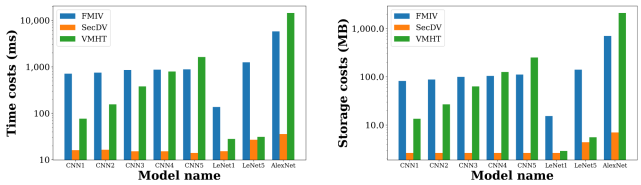
Authorized licensed use limited to: GUANGZHOU UNIVERSITY. Downloaded on October 19,2026 at 08:58:06 UTC from IEEE Xplore. Restrictions apply.



(a) The verification time costs of requesting normal models

(b) The verification storage costs of requesting normal models

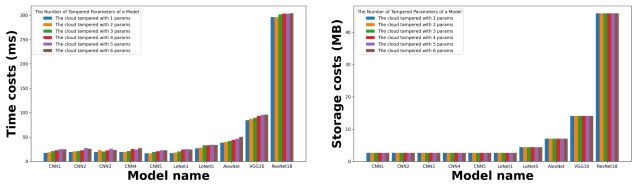
Fig. 5. The verification time and storage costs when the client requests different normal models.



(a) The comparison of verification time costs when the client requests different models

(b) The comparison of verification storage costs when the client requests different models

Fig. 6. The comparison of verification time and storage cost for the client requests different models.



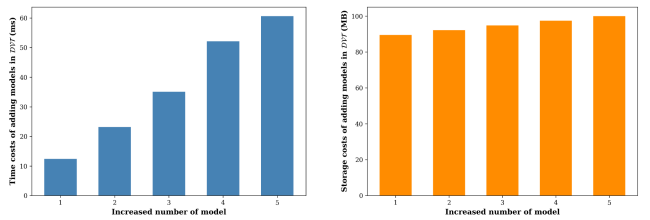
(a) The time costs of detecting tamper models

(b) The storage costs of detecting tamper models

Fig. 7. The time and storage costs when the client detects different tamper models with different tamper parameters.

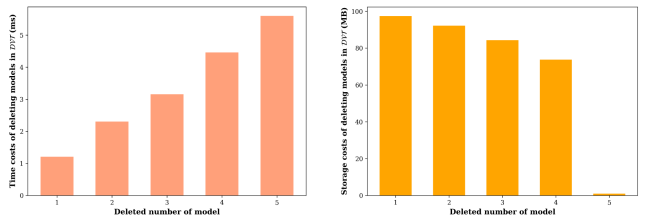
verification time and storage costs than both FMIV and VMHT when verifying different untampered models. This efficiency gain stems from SecDV's use of a chameleon hash function, which allows verification without requiring traversal from the leaf node to the root. Additionally, the model parameters are compressed before verification, further reducing storage overhead. In contrast, FMIV and VMHT rely on Merkle hash trees, which necessitate to complete path verification from leaf to root. Although VMHT improves upon FMIV by using parameter sampling to reduce verification size, it still incurs higher costs than SecDV. For instance, when verifying LeNet1, the verification time and storage cost of SecDV are 15.328 ms and 2.642 MB, respectively, compared to 136.638 ms and 15.288 MB for FMIV, and 28.010 ms and 2.89 MB for VMHT.

Tamper Detection: As shown in Fig. 7(a) and (b), the detection time costs increase proportionally with the number of tampered parameters in a model, while the storage costs remain constant



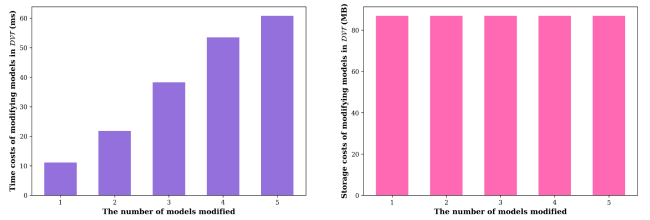
(a) The time costs for model addition

(b) The storage costs for model addition



(c) The time costs for model deletion

(d) The storage costs for model deletion



(e) The time costs for model modification

(f) The storage costs for model modification

Fig. 8. The time and storage costs when the cloud server updates models, e.g., add, delete and modify.

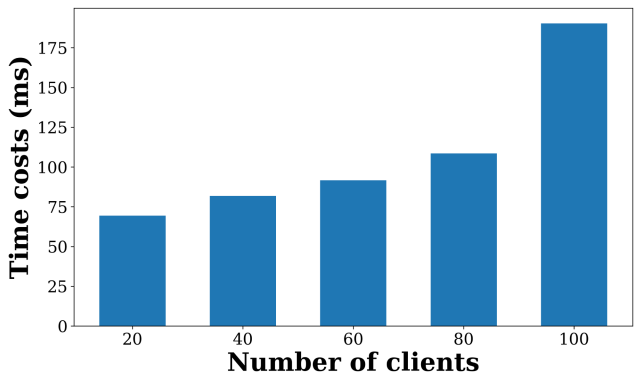


Fig. 9. The verification time costs for different numbers of clients.

regardless of tampering extent. This phenomenon can be attributed to two key factors in our verification framework. First, the increasing time costs correlate directly with tampering severity because the client must meticulously examine and verify the provided proof to identify each tampered parameter precisely. As the number of tampered parameters grows, this verification process requires additional computational effort to locate and validate each compromised value against the established $\mathcal{DVT}$, resulting in a linear increase in detection time. Conversely, the

Authorized licensed use limited to: GUANGZHOU UNIVERSITY. Downloaded on October 19,2026 at 08:58:06 UTC from IEEE Xplore. Restrictions apply.

stability of storage costs stems from our underlying assumption about tampered behavior: malicious cloud servers, when tampering with model parameters, preserve the original structural dimensions of the model. Since the tampering only alters parameter values while maintaining the same parameter shapes and overall model architecture, the storage requirements for the verification framework remain unchanged. This dimensional consistency ensures that our verification mechanism maintains a predictable and efficient storage footprint regardless of the extent of parameter tampering, an important consideration for resource-constrained deployment scenarios.

Update Models: The update operations are performed based on an established $\mathcal{DVT}$, where five new models[3] are added. As shown in Fig. 8(a) and (b), both time and storage costs increase proportionally with the number of added models. When five models are added, the time and storage costs reach 60.670 ms and 100.071MB, respectively. In Fig. 8(c) and (d), it can be observed that the time cost grows linearly with the number of deleted models, while the storage cost decreases accordingly. When five models are deleted, the time and storage costs are 5.501ms and 0.007 MB, respectively. It is worth noting that the storage cost does not drop to zero even after all models are removed from $\mathcal{DVT}$, as the tree structure is preserved to maintain the integrity of the verification framework. Finally, Fig. 8(e) and (f) show that the time cost of modifying model parameters increases linearly with the number of modified models, whereas the storage cost remains constant, regardless of how many models are updated. In summary, the update latency of SecDV remains within the millisecond range, which indicates SecDV is feasible.

Batch Verification: Fig. 9 demonstrates that the verification time increases linearly with the number of clients verifying a model. Even when the number of clients reaches 100, the total verification time remains relatively low at only 190.349 ms. This linear scalability is attributed to the design of SecDV, where each client independently verifies a constant-size verification path without requiring interaction with other clients. Furthermore, the verification process leverages precomputed signatures and chameleon hash values, allowing lightweight and parallelizable verification, which ensures that the overall time cost scales efficiently with the number of clients.

VIII. Conclusion

In this paper, we have proposed a lightweight secure DNN inference service with dynamic verification, named SecDV. First, SecDV employs the CKKS homomorphic encryption scheme to encrypt both input images and model parameters, thereby mitigating the risk of data leakage. Second, SecDV designs a dynamic verification tree $\mathcal{DVT}$ based on the chameleon hash function and digital signature technique, which supports 1) batch verification on the client side; 2) dynamic update of the model parameters, and 3) fine-grained (model parameter-level) tamper detection. Meanwhile, SecDV applies binary compression to process model parameters, compressing multi-dimensional values into a single scalar, which significantly reduces the storage overhead of leaf nodes in $\mathcal{DVT}$. To defend against replay attacks and ensure model version traceability, SecDV also embeds a timestamp and model version number within the $\mathcal{DVT}$. Experimental results also demonstrate that SecDV outperforms existing schemes in terms of performance and verification efficiency. However, SecDV does not yet support the verification of the correctness of the DNN inference computation. As part of our future work, we will investigate the design of a robust verification mechanism to ensure the correctness of the secure DNN inference computation.

[3]These models share the same architecture as CNN1–CNN5.

References

[1] L. Zhang and Y. zhang, "Big data analysis by infinite deep neural networks," *J. Comput. Res. Develop.*, vol. 53, no. 1, pp. 68–79, 2016.

[2] J. Gao, Z. Wang, and X. Wei, "An adaptive pricing framework for real-time AI model service exchange," *IEEE Trans. Netw. Sci. Eng.*, vol. 11, no. 5, pp. 5114–5129, Sep.-Oct. 2024.

[3] J. Ran and D. Li, "A faster privacy-preserving medical image diagnosis scheme with machine learning," *J. Imag. Inform. Med.*, pp. 1–13, 2025.

[4] R. Gilad-Bachrach, N. Dowlin, K. Laine, K. Lauter, M. Naehrig, and J. Wernsing, "CryptoNets: Applying neural networks to encrypted data with high throughput and accuracy," in *Proc. Int. Conf. Mach. Learn.*, 2016, pp. 201–210.

[5] J. W. Bos, K. Lauter, J. Loftus, and M. Naehrig, "Improved security for a ring-based fully homomorphic encryption scheme," in *Proc. 14th IMA Int. Conf. Cryptogr. Coding*, 2013, pp. 45–64.

[6] H. Chabanne, W. A. De, J. Milgram, C. Morel, and E. Prouff, "Privacy-preserving classification on deep neural network," *Cryptol. ePrint Arch.*, pp. 1–18, 2017.

[7] P. Mishra, R. Lehmkuhl, A. Srinivasan, W. Zheng, and R. A. Popa, "Delphi: A cryptographic inference service for neural networks," in *Proc. 29th USENIX Secur. Symp.*, 2020, pp. 2505–2522.

[8] Y. Lindell and B. Pinkas, "A proof of security of Yao's protocol for two-party computation," *J. Cryptol.*, vol. 22, pp. 161–188, 2009.

[9] X. Liu, B. Wu, X. Yuan, and X. Yi, "Leia: A lightweight cryptographic neural network inference system at the edge," *IEEE Trans. Inf. Forensics Secur.*, vol. 17, pp. 237–252, 2022.

[10] J. H. Cheon, A. Kim, M. Kim, and Y. Song, "Homomorphic encryption for arithmetic of approximate numbers," in *Proc. 23rd Int. Conf. Theory Appl. Cryptol. Inf. Secur.*, 2017, pp. 409–437.

[11] D. Li, Q. Lü, X. Liao, T. Xiang, J. Wu, and J. Le, "AVPMIR: Adaptive verifiable privacy-preserving medical image retrieval," *IEEE Trans. Dependable Secure Comput.*, vol. 21, no. 5, pp. 4637–4651, Sep.-Oct. 2024.

[12] D. Johnson, A. Menezes, and S. Vanstone, "The elliptic curve digital signature algorithm (ECDSA)," *Int. J. Inf. Secur.*, vol. 1, pp. 36–63, 2001.

[13] E. Hesamifard, H. Takabi, M. Ghasemi, and R. N. Wright, "Privacy-preserving machine learning as a service," in *Proc. Privacy Enhancing Technol.*, 2018, pp. 123–142.

[14] Q. Zhang, C. Wang, H. Wu, C. Xin, and T. V. Phuong, "GELU-Net: A globally encrypted, locally unencrypted deep neural network for privacy-preserved learning," in *Proc. 27th Int. Joint Conf. Artif. Intell.*, 2018, pp. 3933–3939.

[15] X. Ma, X. Chen, and X. Zhang, "Non-interactive privacy-preserving neural network prediction," *Inf. Sci.*, vol. 481, pp. 507–519 2019.

[16] J. Wan, D. Li, J. Fang, and Z. Jiang, "LPFHE: Low-complexity polynomial CNNs for secure inference over FHE," in *Proc. Eur. Symp. Res. Comput. Secur.*, 2024, pp. 403–423.

[17] R. Dathathri et al., "CHET: An optimizing compiler for fully homomorphic neural-network inferencing," in *Proc. 40th ACM SIGPLAN Conf. Program. Lang. Des. Implementation*, 2019, pp. 142–156.

[18] R. Dathathri, B. Kostova, O. Saarikivi, W. Dai, K. Laine, and M. Musuvathi, "EVA: An encrypted vector arithmetic language and compiler for efficient homomorphic computation," in *Proc. 41th ACM SIGPLAN Conf. Program. Lang. Des. Implementation*, 2020, pp. 546–561.

[19] S. Chowdhary, W. Dai, K. Laine, and O. Saarikivi, "EVA improved: Compiler and extension library for CKKS," in *Proc. 9th Workshop Encrypted Comput. Appl. Homomorphic Cryptogr.*, 2021, pp. 43–55.

[20] R. Lehmkuhl, P. Mishra, A. Srinivasan, and R. Popa, "Muse: Secure inference resilient to malicious clients," *Proc. 30th USENIX Secur. Symp.*, 2021, pp. 2201–2218.

Authorized licensed use limited to: GUANGZHOU UNIVERSITY. Downloaded on October 19,2026 at 08:58:06 UTC from IEEE Xplore. Restrictions apply.

[21] A. Brutzkus, R. Gilad-Bachrach, and O. Elisha, "Low latency privacy preserving inference," in *Proc. Int. Conf. Mach. Learn.*, 2019, pp. 812–821.
[22] Q. Lou, W. J. Lu, C. Hong, and L. Jiang, "Falcon: Fast spectral inference on encrypted data," in *Proc. Adv. Neural Inf. Process. Syst.*, 2020, vol. 33, pp. 2364–2374.
[23] S. Bian, T. Wang, M. Hiromoto, Y. Shi, and T. Sato, "ENSEI: Efficient secure inference via frequency-domain homomorphic convolution for privacy-preserving visual recognition," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2020, pp. 9403–9412.
[24] S. Li et al., "FALCON: A Fourier transform based approach for fast and secure convolutional neural network predictions," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2020, pp. 8705–8714.
[25] F. Bourse, M. Minelli, M. Minihold, and P. Paillier, "Fast homomorphic evaluation of deep discretized neural networks," in *Proc. 38th Annu. Int. Cryptol. Conf.*, 2018, pp. 483–512.
[26] I. Chillotti, M. Joye, and P. Paillier, "Programmable bootstrapping enables efficient homomorphic inference of deep neural networks," in *Proc. 5th Int. Symp. Cyber Secur. Cryptogr. Mach. Learn.*, 2021, pp. 1–19.
[27] I. Chillotti, N. Gama, M. Georgieva, and M. Izabachéne, "Faster packed homomorphic operations and efficient circuit bootstrapping for TFHE," in *Proc. Int. Conf. Theory Appl. Cryptol. Inf. Secur.*, 2017, pp. 377–408.
[28] W. Yang, X. Wang, Z. Guan, L. Wu, X. Du, and M. Guizani, "SecureSL: A privacy-preserving vertical cooperative learning scheme for web 3.0," *IEEE Trans. Netw. Sci. Eng.*, vol. 11, no. 5, pp. 3983–3994, Sep.-Oct. 2024.
[29] K. Cheng et al., "Manto: A practical and secure inference service of convolutional neural networks for IoT," *IEEE Internet Things J.*, vol. 10, no. 16, pp. 14856–14872, Aug. 2023.
[30] Q. Zhang, C. Xin, and H. Wu, "SecureTrain: An approximation-free and computationally efficient framework for privacy-preserved neural network training," *IEEE Trans. Netw. Sci. Eng.*, vol. 9, no. 1, pp. 187–202, Jan./Feb. 2022.
[31] X. Liu, Y. Zheng, X. Yuan, and X. Yi, "Securely outsourcing neural network inference to the cloud with lightweight techniques," *IEEE Trans. Dependable Secure Comput.*, vol. 20, no. 1, pp. 620–636, Jan.-Feb. 2023.
[32] X. Yang, J. Chen, K. He, H. Bai, C. Wu, and R. Du, "Efficient privacyp-reserving inference outsourcing for convolutional neural networks," *IEEE Trans. Inf. Forensics Secur.*, vol. 18, pp. 4815–4829, 2023.
[33] K. Huang, X. Liu, S. Fu, D. Guo, and M. Xu, "A lightweight privacy-preserving CNN feature extraction framework for mobile sensing," *IEEE Trans. Dependable Secure Comput.*, vol. 18, no. 3, pp. 1441–1455, May-Jun. 2021.
[34] M. Li, S. Chow, S. Hu, Y. Yan, C. Shen, and Q. Wang, "Optimizing privacy-preserving outsourced convolutional neural network predictions," *IEEE Trans. Dependable Secure Comput.*, vol. 19, no. 3, pp. 1592–1604, May-Jun. 2022.
[35] S. Fei, Z. Yan, H. Xie, and G. Liu, "Sec-E2E: End-to-end communication security in LS-HetNets based on blockchain," *IEEE Trans. Netw. Sci. Eng.*, vol. 11. no. 1, pp. 761–778, Jan.-Feb. 2024.
[36] J. Wang, D. He, A. Castiglione, B. Gupta, M. Karuppiah, and L. Wu, "PCNN_CEC: Efficient and privacy-preserving convolutional neural network inference based on cloud-edge-client collaboration," *IEEE Trans. Netw. Sci. Eng.*, vol. 10, no. 5, pp. 2906–2923, Sep.-Oct. 2023.
[37] R. Zhao, Y. Xie, D. He, K. Choo, and Z. Jiang, "Pratical privacy-preserving convolutional neural network inference framework with edge computing for health monitoring," *IEEE Trans. Netw. Sci. Eng.*, vol. 11, no. 6, pp. 5995–6006, Nov.-Dec. 2024.
[38] B. Wang et al., "Privacy-preserving convolutional neural network classification scheme with multiple keys," *IEEE Trans. Serv. Comput.*, vol. 17, no. 1, pp. 322–335, Jan.-Feb. 2024.
[39] C. Gao and J. Yu, "SecureRC: A system for privacy-preserving relation classification using secure multi-party computation," *Comput. Secur.*, vol. 128, 2023, Art. no. 103142.
[40] L. Chen, D. Xiao, X. Xiao, and Y. Zhang, "Secure and efficient federated learning via novel authenticable multi-party computation and compressed sensing," *IEEE Trans. Inf. Forensics Secur.*, vol. 19, pp. 10141–10156, 2024.
[41] G Cui, Q. He, B. Li, X. Xia, and F. Chen, "Efficient verification of edge data integrity in edge computing environment," *IEEE Trans. Serv. Comput.*, vol. 15, no. 6, pp. 3233–3244, Nov.-Dec. 2022.
[42] G. Ateniese et al., "Provable data possession at untrusted stores," in *Proc. 14th ACM Conf. Comput. Commun. Secur.*, 2007, pp. 598–609.
[43] H. Wang, Y. Lin, and X. Fu, "A lightweight data integrity verification with data dynamics for mobile edge computing," *Secur. Commun. Netw.*, vol. 2022, no. 1, 2022, Art. no. 1870779.
[44] N. Zhang et al., "From data integrity to global model integrity for federated learning: An MHT-based approach," in *Proc. IEEE Glob. Commun. Conf.*, 2024, pp. 956–961.
[45] B. Li, F. Chen, H. Jin, Y. Xiang, and Y. Yang, "Auditing cache data integrity in the edge computing environment," *IEEE Trans. Parallel Distrib. Syst.*, vol. 32, no. 5, pp. 1210–1223, May 2021.
[46] T. Liu, X. Xie, and Y. Zhang, "ZKCNN: Zero knowledge proofs for convolutional neural network predictions and accuracy," in *Proc. 2021 ACM SIGSAC Conf. Comput. Commun. Secur.*, 2021, pp. 2968–2985.
[47] J. Weng, J. Weng, G. Tang, A. Yang, M. Li, and J. N. Liu, "pvCNN: Privacy-preserving and verifiable convolutional neural network testing," *IEEE Trans. Inf. Forensics Secur.*, vol. 18, pp. 2218–2233, 2023.
[48] A. Riasi, J. Guajardo, and T. Hoang, "Privacy-preserving verifiable neural network inference service," in *Proc. Annu. Comput. Secur. Appl. Conf.*, 2024, pp. 683–698.
[49] S. Lee, H. Ko, J. Kim, and H. Oh, "vCNN: Verifiable convolutional neural network based on zk-SNARKs," *IEEE Trans. Dependable Secure Comput.*, vol. 21, no. 4, pp. 4254–4270, Jul.-Aug. 2024.
[50] N. Attrapadung, G. Hanaoaka, R. Hiromasa, Y. Koseki, T. Matsuda, and Y. Nishida, "Privacy-preserving verifiable CNNs," in *Proc. Int. Conf. Appl. Cryptogr. Netw. Secur.*, 2024, pp. 373–402.
[51] V. Chandrasekaran, K. Chaudhuri, I. Giacomelli, S. Jha, and S. Yan, "Exploring connections between active learning and model extraction," in *Proc. 29th USENIX Secur. Symp.*, 2020, pp. 1309–1326.
[52] R. N. Reith, T. Schneider, and O. Tkachenko, "Efficiently stealing your machine learning models," in *Proc. 18th ACM Workshop Privacy Electron. Soc.*, 2019, pp. 198–210.
[53] M. Juuti, S. Szyller, S. Marchal, and N. Asokan, "Prada: Protecting against DNN model stealing attacks," in *Proc. IEEE Eur. Symp. Secur. Privacy*, 2019, pp. 512–527.
[54] H. Jia, C. A. Choquette-Choo, V. Chandrasekaran, and N. Papernot, "Entangled watermarks as a defense against model extraction," in *Proc. 30th USENIX Secur. Symp.*, 2021, pp. 1937–1954.
[55] H. Hu and J. Pang, "Stealing machine learning models: Attacks and countermeasures for generative adversarial networks," in *Proc. Annu. Comput. Secur. Appl. Conf.*, 2021, pp. 1–16.
[56] M. Mazeika, B. Li, and D. Forsyth, "How to steer your adversary: Targeted and efficient model stealing defenses with gradient redirection," in *Proc. Int. Conf. Mach. Learn.*, 2022, pp. 15241–15254.
[57] T. Orekondy, B. Schiele, and M. Fritz, "Prediction poisoning: Towards defenses against DNN model stealing attacks," in *Proc. Int. Conf. Representation Learn.*, pp. 1–17, 2020.
[58] A. Ibarrondo and A. Viand, "Pyfhel: PYthon for homomorphic encryption libraries," in *Proc. 9th Workshop Encrypted Comput. Appl. Homomorphic Cryptogr.*, 2021, pp. 11–16.
[59] H. Kario and B. Warner, "Pure-python ECDSA signature/verification and ECDH key agrrement," 2021. [Online]. Available: https://github.com/tlsfuzzer/python-ecdsa
[60] Y. LeCun, "The MNIST database of handwritten digits," 1998. [Online]. Available: http://yann.lecun.com.exdb.mnist/
[61] K. Alex, G. Hinton, and N. Vinod, "CIFAR10 (Canadian Institute for Advanced Research)," 2009. [Online]. Available: https://www.cs.toronto.edu/kriz/cifar.html



Dong Li (Graduate Student Member, IEEE)received the M.S. degree in signal and information processing from Southwest University, Chongqing, China, in 2021. He is currently working toward the Ph.D. degree in computer science and technology with Chongqing University, Chongqing, China. He is also a Visiting Scholar with Nanyang Technological University, Singapore. His research interests include searchable encryption, privacy-preserving, and machine learning.

Authorized licensed use limited to: GUANGZHOU UNIVERSITY. Downloaded on October 19,2026 at 08:58:06 UTC from IEEE Xplore. Restrictions apply.



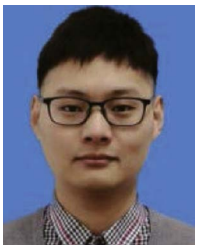
Anupam Chattopadhyay (Senior Member, IEEE) received the B.E. degree from Jadavpur University, Kolkata, India, in 2000, the M.Sc. degree from ALaRI, Lugano, Switzerland, in 2002, and the Ph.D. degree from RWTH Aachen, Aachen, Germany, in 2008. He has held Visiting positions with Politecnico di Torino, Turin, Italy, EPFL, Lausanne, Switzerland, Technion, Haifa, Israel, Indian Statistical Institute, Kolkata, and Kyoto University, Kyoto, Japan. In 2014, he was appointed as an Assistant Professor with Nanyang Technological University, Singapore, where he was promoted to Associate Professor with Tenure in 2019. His research interests include cyber security, computer architecture, electronic design automation, and emerging technologies. Prof. Chattopadhyay is also a fellow of JSPS and Inter-Continental Academia and a senior member of ACM.



Yang Zhou received the master's degree from the South China University of Technology,Guangzhou, China, in 2019. In 2019, he joined Tencent's WeChat team, where he currently an Engineer, specializing in the development of WeChat Channel's recommendation system and other Algorithm Engineering Projects. His research interests include deep learning, natural language processing (NLP), and AI inference/training infrastructure.



Bowen Xing received the B.S. degree in data science and Big Data technology from the Wuhan University of Technology, Wuhan, China, in 2022. He is currently working toward the M.S. degree in computer science from Chongqing University, Chongqing, China. His research interests include federated learning, privacy protection, and privacy machine learning.



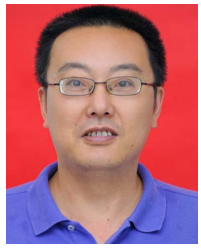
Qingguo Lü (Member, IEEE) received the M.S. and Ph.D. degrees in signal and information processing and computational intelligence and information processing from Southwest University, Chongqing, China, in 2018 and 2021, respectively. He is currently a Postdoctor and a Hongshen Young Teacher (special support) with the College of Computer Science, Chongqing University, Chongqing, China. His research interests include privacy protection, distributed optimization, machine learning, and smart grids. Dr. Lü was the recipient of the Outstanding Ph.D. Thesis Award of ACM China (Chongqing Branch) in 2021, and the Outstanding Master's Thesis Award of Chongqing, in 2019. He is also the Guest Editor of the *Applied Sciences* (SI: Advanced Artificial Intelligence Theories and Applications.



Jiahui Wu received the Ph.D. degrees in signal and information processing and computational intelligence and information processing from Southwest University, Chongqing, China, in 2021, respectively. She is currently an Assistant Researcher with Peng Cheng Laboratory, Shenzhen, China. Her research interests include information security, cloud computing security, and data mining.



Tao Xiang (Senior Member, IEEE) received the B.Eng., M.S., and Ph.D. degrees in computer science from Chongqing University, Chongqing, China, in 2003, 2005, and 2008, respectively. He is currently a Professor with the College of Computer Science, Chongqing University. His research interests include multimedia security, cloud security, data privacy, and cryptography. He has authored or coauthored more the 100 papers on international journals and conferences. He was also a Referee for numerous international journals and conferences.



Xiaofeng Liao (Fellow, IEEE) received the B.S. and M.S. degrees in mathematics from Sichuan University, Chengdu, China, in 1986 and 1992, respectively, and the Ph.D. degree in circuits and systems from the University of Electronic Science and Technology of China, Chengdu, in 1997. From 1999 to 2012, he was a Professor with Chongqing University, Chongqing, China. From 2012 to 2018, he was a Professor and the Dean with the College of Electronic and Information Engineering, Southwest University, Chongqing. He is currently a Professor and the Dean of the College of Computer Science, Chongqing University. He is also a Yangtze River Scholar with the Ministry of Education of China, Beijing, China. From 1997 to 1998, he was a Research Associate with the Chinese University of Hong Kong, Hong Kong. From 1999 to 2000, he was a Research Associate with the City University of Hong Kong, Hong Kong. From 2001 to 2001 and 2002 to 2002, he was a Senior Research Associate with the City University of Hong Kong, Hong Kong. From 2006 to 2007, he was a Research Fellow with the City University of Hong Kong. He also holds five patents, and has authored or coauthored four books and more than 300 international journal and conference papers. His research interests include decentralized optimization, machine learning, neural networks, and bifurcation and chaos cryptography. Dr. Liao is also an Associate Editor for IEEE TRANSACTIONS ON CYBERNETICS and IEEE TRANSACTIONS ON NEURAL NETWORKS AND LEARNING SYSTEMS.

Authorized licensed use limited to: GUANGZHOU UNIVERSITY. Downloaded on October 19,2026 at 08:58:06 UTC from IEEE Xplore. Restrictions apply.